UET AI Coding Arena

HỆ THỐNG TỔ CHỨC GIẢI ĐẤU BOT PYTHON

Báo cáo kỹ thuật và hướng dẫn triển khai
Môn Lập trình mạng - UET, Đại học Quốc gia Hà Nội
Bản bàn giao: 25/10/2026
Nhánh mã nguồn: integrate-safe-enhancements

Mục tiêu

Hệ thống tổ chức giải có chia bảng và vòng loại trực tiếp. Sinh viên nộp mã Python; bộ chấm thực thi có giới hạn; ban tổ chức (BTC) cấu hình, giám sát và công bố kết quả. Trường sở hữu tên miền, máy chủ và dữ liệu. Vercel được giữ riêng, không phải thành phần bắt buộc của mô hình triển khai chính thức.

Phạm vi bàn giao

Website sinh viên/BTC, API xác thực và nhận bài, PostgreSQL lưu hàng đợi/kết quả, worker Docker dùng CPython WASI/Wasmtime, bot mẫu, replay offline, đấu tập Elo, dashboard vận hành, backup/restore và bộ kiểm thử tải có thể tái lập. Bộ nộp không chứa tài khoản thật, token, mật khẩu hoặc backup database.

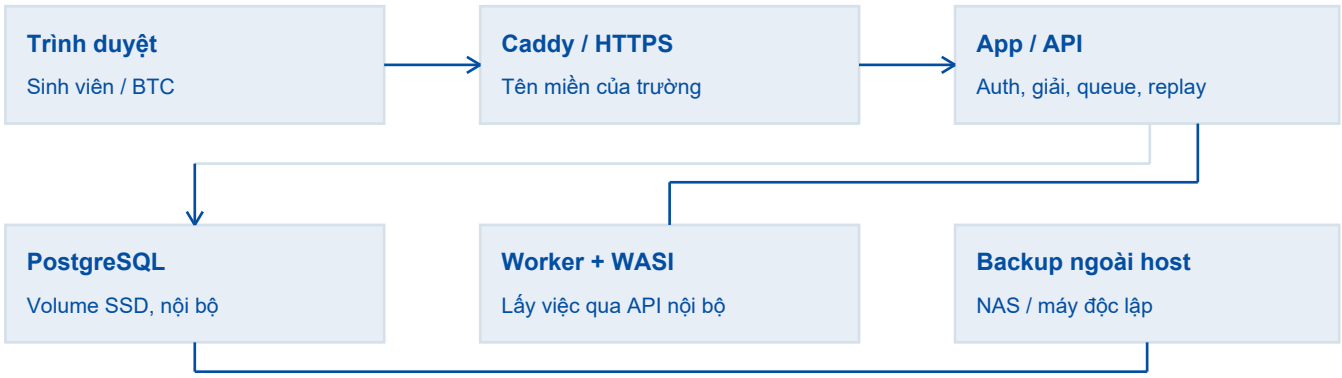
Bốn tiêu chí thiết kế

Bảo mật: session, membership và mã mời đội riêng; kiểm tra quyền phía server. An toàn: Python không tin cậy chạy trong WASI có quyền tối thiểu và giới hạn tài nguyên. Ổn định: hàng đợi bền vững, lease/heartbeat và commit idempotent. Tiết kiệm: hạ tầng trường, PostgreSQL kiêm hàng đợi, tăng worker theo nhu cầu đo được.

Trò chơi hiện hành là FRONTLINE trên bàn 9×9 theo engine dự án, không phải oẳn-tù-tì một lượt. Luật và giới hạn được chốt trước khi mở nhận bài. Báo cáo phân biệt kết quả đã kiểm tra với cấu hình cần hoàn tất tại trường.

1. Kiến trúc và infrastructure

Trình duyệt → HTTPS/Caddy → app Next.js → PostgreSQL.
Worker → API nội bộ lấy lease → engine tin cậy + WASI riêng cho bot → API xác minh kết quả → transaction → bảng điểm/replay.



Tại mỗi máy

Máy sinh viên: trình duyệt, Python/IDE tùy chọn để viết bot. Không cần Docker để tham gia. BTC thao tác admin panel trên trình duyệt; người vận hành máy chủ mới cần công cụ triển khai.

Máy chủ trường: Caddy nhận HTTPS 80/443; app phục vụ giao diện, session, phân quyền, nhận bài, quản lý giải và replay. PostgreSQL trên volume SSD lưu tài khoản, membership, source/SHA, cấu hình, snapshot, jobs, kết quả và audit. Database không mở cổng ra Internet.

Máy chấm: có thể cùng host ở pilot hoặc host riêng trong LAN/VPN. Worker chủ động gọi API bằng token, không có mật khẩu database. Supervisor điều khiển engine và sandbox riêng cho hai bot. Guest không được cấp token. Máy/NAS backup giữ bản sao độc lập, không chỉ snapshot trên cùng ổ đĩa.

Luồng nhận bài ngắn

API lưu source/hash, validation job và audit trong một transaction rồi trả kết quả tiếp nhận; không chờ Python chạy trong request. HTTP thành công nghĩa là đã nhận và xếp hàng, chưa có nghĩa bot hợp lệ hoặc thi đấu xong. Đấu tập Elo dùng bảng ops_* riêng, không sửa điểm giải chính thức; công việc chính thức được lấy trước việc nền tương ứng.

Ánh xạ vào mã nguồn

competition/vercel.mjs: API và nối worker; competition/store.mjs cùng adapter PostgreSQL: giải/transaction; judge/worker.mjs: lease/heartbeat; runner.mjs và sandbox.py: thực thi; operations/: telemetry, Elo, self-test và review; deploy/competition-schema.sql: schema; app/competition, app/operations: giao diện. Hướng dẫn PostgreSQL local là profile chính; tài liệu SQLite/Vercel cũ là lựa chọn độc lập.

2. Bảo mật và an toàn

Danh tính và quyền sở hữu bài

Đội được suy ra từ session → membership, không tin tên đội do browser gửi. BTC đối chiếu danh sách, gắn email/MSSV và phát mã mời entropy 192 bit, lưu hash, dùng một lần. API quản trị kiểm tra quyền phía server; POST kiểm tra Origin/CSRF. Source và SHA-256 được lưu bất biến.

Lọc @vnu.edu.vn không chứng minh chủ hộp thư. Phiên bản này không có OTP/SSO; có thể xảy ra đăng ký trước email của người khác. BTC phải kiểm tra ngoài hệ thống khi tranh chấp. Nên bổ sung SSO hoặc xác minh email trước khi dùng email làm định danh chính thức.

Git là đường nhập bot.py từ repo công khai đã gắn đội và commit đầy đủ đã duyệt. Commit author tự khai không chứng minh danh tính sinh viên. Không bắt buộc Git; tải .py vẫn hoạt động khi không có Internet.

Ranh giới thực thi

Python chạy trong CPython WebAssembly/Wasmtime, không chạy bằng Python máy chủ. Guest không được cấp mạng, secret, thư mục database hoặc quyền ghi host. Docker bổ sung read-only filesystem, cap_drop, no-new-privileges, giới hạn PID/CPU/RAM và mạng judge nội bộ. Sandbox giảm rủi ro, không thay cập nhật host/runtime hay kiểm định độc lập.

Source tối đa 64 KiB; giới hạn startup/nước đi, fuel, bộ nhớ Wasm và output. Engine hỗ trợ maxPlies 20-200, move timeout 100-5.000 ms, memory 64-256 MiB. WASI không hỗ trợ pip/native extension/socket/subprocess. E01-E06 phân loại lỗi cú pháp, giao diện, nước sai, timeout, khả năng không hỗ trợ và tài nguyên/output.

Kết quả và tương đồng

Engine tin cậy kiểm tra luật/nước đi/replay; bot không tự khai điểm. Review AST chỉ parse source có giới hạn, không import/chạy bài; tối đa 100 bài hợp lệ mới nhất và báo rõ khi cắt bớt. Đây là gợi ý cho BTC, không tự kết tội hay trừ điểm. Không tuyên bố chống mọi hình thức thông đồng hoặc sao chép.

3. Ổn định, tốc độ, tiết kiệm

Nộp đồng thời

Đội khác nhau ghi đồng thời; cùng đội tuần tự qua khóa hàng để giữ hạn mức. Source, validation job và audit được ghi cùng transaction. Gửi lại source giống nhau trả về cùng bài, không tạo job trùng. Hạn mức: 5 giây/phiên bản mới, 2 pending, 200 phiên bản/đội. Quá tải trả Retry-After; client cần chờ, không gửi dồn liên tục.

Phục hồi và tính nhất quán

Job có lease 90 giây, heartbeat và token fencing. Mất worker thì việc có thể được cấp lại; kết quả lease cũ bị từ chối. Gửi trùng kết quả không cộng điểm thêm. Lỗi hạ tầng thử tối đa 3 lần rồi chờ BTC xử lý; lỗi bot tạo verdict ở đúng phía.

Freeze sau hạn và sau khi hết validation pending; chọn bản hợp lệ mới nhất trước hạn, cố định source/hash/seed/config. Engine/runtime không khớp phải tạm dừng. Replay và audit phục vụ đối soát; không âm thầm áp luật mới hoặc sửa điểm đã công bố.

Vì sao tiết kiệm

Local tránh quota theo lượt gọi Vercel/Neon. PostgreSQL làm hàng đợi, chưa cần Redis/Kafka/Kubernetes. Chạy ít worker khi rảnh, tăng theo backlog và CPU/RAM đo được. Đấu tập có hạn mức. Chia bảng tránh full round-robin giữa 4.000 đội; vẫn cần tính số trận theo thể thức thực tế.

Local không có nghĩa chi phí bằng 0: còn máy chủ, điện, mạng, SSD, backup và nhân lực trực. API nhận nhanh không chứng minh chấm nhanh; tốc độ drain phụ thuộc độ nặng bot và worker. Tăng worker không giải quyết mọi nghẽn database.

Điểm lỗi và cách xử lý

Một host/DB chính vẫn là điểm lỗi đơn, chưa có HA tự failover: cần UPS, backup ngoài host và máy dự phòng. Đĩa đầy/khóa chờ: giám sát dung lượng, pool và backlog; giữ retry hữu hạn. Control/worker còn khóa ghi chung; tối ưu khóa theo job/giải khi đo được nghẽn. Backup phải thử restore vào DB riêng. Giữ image tương thích và đường rollback khi nâng cấp.

4. Hướng dẫn setup tại trường

Điểm xuất phát cho pilot: Linux, SSD, RAM 16 GiB, khoảng 8-12 CPU logic, rồi đo lại. Đây là gợi ý vận hành, không cam kết cho 4.000 người đồng thời. Windows dùng Docker Desktop/Linux containers. Setup cần Git, Docker Compose, Node 22.16+; lần build đầu cần Internet.

1. Lấy source và khởi động

Clone repository, checkout integrate-safe-enhancements. Chạy trong thư mục dự án:
npm ci
npm run local:init
docker compose -f compose.local.yaml up -d --build --wait

local:init tạo secret riêng, không ghi đè secret đã có. Khi dùng volume cũ phải giữ đúng secret cũ. Trên Linux cấp quyền đọc URL/token cho UID 1000 theo deploy/LOCAL_POSTGRES.md. Web mặc định chỉ bind 127.0.0.1:3000; schema được khởi tạo khi app chạy.

2. Tạo tài khoản BTC

Đăng ký tài khoản VNU trên website, sau đó chạy:
docker compose -f compose.local.yaml exec app node scripts/local-admin.mjs your@vnu.edu.vn
Đăng xuất/đăng nhập lại; kiểm tra /competition, /operations và /practice. Không có admin password dùng chung trong bộ nộp.

3. Đưa lên tên miền và backup

Trường cấp tên miền/DNS, IP/NAT/firewall 80/443. Tạo .env.campus chứa ARENA_DOMAIN thật, chạy:
docker compose -f compose.local.yaml -f deploy/compose.local-public.yaml --env-file .env.campus up -d --build --wait

Proxy công khai chặn /api/judge; worker cùng host dùng mạng nội bộ, host khác cần endpoint riêng qua HTTPS/VPN. DB không công khai. Kiểm tra chứng thư và Origin từ mạng ngoài. Khi đã có image/runtime, tải .py và chấm local không cần cloud; GitHub import vẫn cần Internet.

Chạy npm run local:backup và node tests/local-backup.test.mjs. Cài deploy/uet-arena-backup.service/.timer trên Linux, sửa User/đường dẫn/Node, bật timer và kiểm tra log. Sao chép dump/secret sang nơi riêng được bảo vệ. DNS/TLS trường và timer chưa được cài tại trường trong lần bàn giao này. Vercel cũ được giữ nguyên.

5. Tổ chức và sử dụng giải đấu

Quy trình BTC và sinh viên

1. BTC tạo giải nháp, chốt luật/seed/runtime, deadline, số bảng và suất đi tiếp. Tổng số đội knockout phải là lũy thừa của hai; công khai tie-break trước giải.

2. Nhập roster, đối chiếu danh tính, phát mã mời riêng gắn email/MSSV. Sinh viên đăng nhập, nhận đúng đội và nộp .py hoặc commit Git đã duyệt.

3. Theo dõi version/hash và pending/valid/invalid. Validation worker chính thức là chuẩn tương thích; kết quả browser practice không thay chuẩn chấm.

4. Sau hạn, đợi validation hoàn tất rồi freeze. Hệ thống seeded shuffle/zíc zắc chia bảng, chạy các cặp đổi bên và vòng loại trực tiếp. BTC theo dõi failed jobs, heartbeat, backlog và replay.

5. Review nghi vấn, kiểm tra replay, publish và export. Lưu audit/backup phục vụ đối soát; không sửa điểm trực tiếp.

Thể thức hiện hành

Mỗi cặp hai ván đổi bên. Thắng ván 1 điểm, hòa 0,5; thắng/hòa/thua cặp 3/1/0. Xếp bảng dùng mini-league đối đầu, hiệu số ván rồi seed. Knockout hòa có extraSets; vẫn hòa thì seed nhỏ hơn đi tiếp. Các quy tắc phải được công bố trước giải.

Công cụ vận hành và hỗ trợ

/operations hiển thị queue, worker, readiness, self-test chạy sandbox, bằng chứng backup/restore và review AST. HTTPS readiness phản ánh cấu hình, không xác minh chứng thư từ ngoài. /practice có Elo riêng K=24, hai ván đổi bên và matching có hạn mức; không sửa standings chính thức. /load-demo là trang chỉ đọc dành riêng profile thử tải.

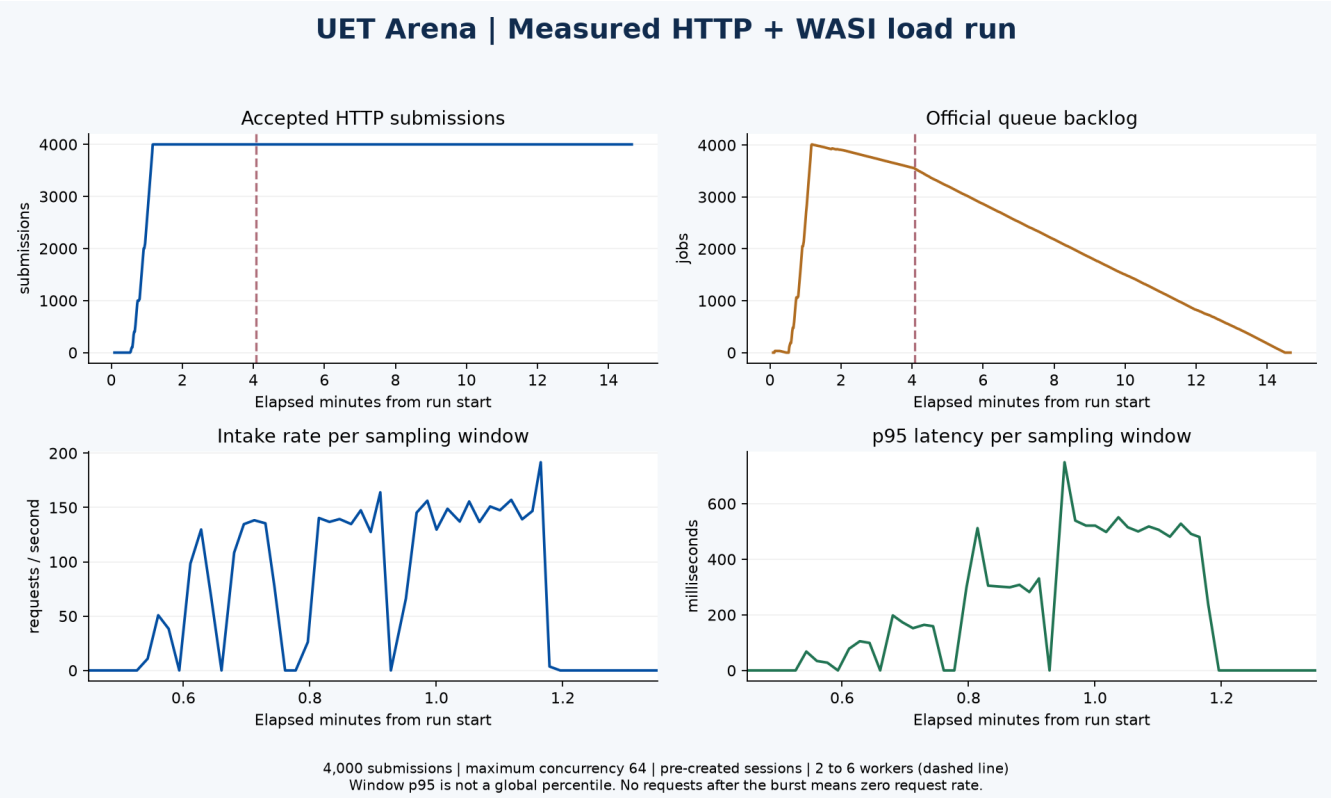
Origin sai: sửa PUBLIC_ORIGIN khớp URL. Worker im lặng: kiểm tra token, mạng và runtime. Pending tăng: kiểm tra CPU/RAM rồi scale theo tài nguyên host. Invalid: đọc E01-E06. Dữ liệu thiếu sau restart: kiểm tra volume/profile/secret. Dừng bằng stop, không dùng down -v để dừng thông thường.

6. Kiểm thử tải thực tế

Phiên 859f0c413587, đo ngày 24/10/2026 giờ Việt Nam. Profile Docker riêng, máy phát tải cùng host. App giới hạn 2 CPU/1,5 GiB; DB 1 CPU/768 MiB; mỗi worker 1 CPU/1 GiB. Pool tăng từ 2 lên 6 khi backlog cao; đường đứt trên biểu đồ đánh dấu thời điểm scale.

HTTP: 4.000/4.000 bài, 0 lỗi, 0 client retry. Tăng concurrency 1/8/16/32/64; burst khoảng 39,1 giây gồm tăng tải và can thiệp worker. RPS cửa sổ cao nhất 191,6; p95 cửa sổ lớn nhất 749 ms, không phải percentile toàn bộ request.

Queue đạt đỉnh 4.013 job; cuối phiên hết official backlog. Có 4.032 validation done: 4.024 valid và 8 invalid dự kiến (gồm 32 bài giải nhỏ). Giải riêng 32 đội hoàn tất 126 game/replay xác minh. Một worker được dừng/bật lại; Elo giữ nguyên qua restart app. Toàn phiên gồm chuẩn bị, backup, giải, drain và restart: 14,65 phút; có 1 lỗi lấy telemetry, không phải lỗi nộp bài.



Đây là 4.000 bài với tối đa 64 request đồng thời, session tạo trước, không phải 4.000 người đồng thời. Không đo mass login, toàn bài 64 KiB hoặc full giải 4.000 bot. Có đăng ký VNU qua HTTP riêng; không lấy đó làm throughput đăng nhập. Không suy ra thời gian chấm mọi giải từ RPS nhận bài.

7. Nghiệm thu và bàn giao

Bằng chứng và giới hạn

Bộ test bao gồm engine, account policy, sandbox, replay, competition API, operations API/PostgreSQL trong schema tạm và AST không chạy source. WASI được kiểm tra với bot lỗi/nguy hiểm. Phiên HTTP thật hoàn tất nhận bài, drain, giải nhỏ và độ bền Elo qua restart. Evidence JSON và biểu đồ không chứa token/cookie; không nộp fixture login hoặc database dump.

Ngày 25/10, sau khi khôi phục Docker, web và hai worker hoạt động; chạy lại PostgreSQL/API đều qua. Backup sau đợt tải đã restore vào database riêng và đối chiếu số bản ghi khớp cả dữ liệu giải lẫn ops_bots/ratings/pairs/jobs. Evidence restore-after-load.json phản ánh tổng database, gồm các lần thử trước; không phải số riêng phiên tải. Kiểm tra số bản ghi không thay thế đối chiếu mọi giá trị dữ liệu.

Chưa chứng minh 4.000 người đồng thời đăng nhập/polling, source lớn đồng loạt, full tournament 4.000 bot, DDoS Internet, HA tự failover hoặc an toàn tuyệt đối. Review AST chỉ là hỗ trợ; email chưa xác minh. Thời gian giải phụ thuộc runtime và lịch đấu thực tế.

Cần nghiệm thu tại trường

Dùng máy phát tải riêng, đo mass login/polling, source lớn và bot sát giới hạn. Thử DB restart, hết đĩa, mất điện, lease expiry, restore ngoài host và rollback. Kiểm tra DNS/TLS/firewall, retention và quy trình danh tính. Nên bổ sung SSO/email verification trước khi dùng email làm định danh chính thức.

TypeScript và API đã kiểm tra; kiểm tra trực quan giao diện desktop/mobile chưa có bằng chứng đầy đủ. Cần kiểm tra thủ công /competition, /operations, /practice trước ngày thi. Biểu đồ đo là kết quả thực nghiệm, không phải ảnh chụp giao diện hoặc bằng chứng nghiệm thu bố cục.

Bộ nộp

BAO_CAO_UET_ARENA.pdf: báo cáo này. REPORT.md: nguồn chỉnh sửa. README.md và DEMO.md: setup, kịch bản trình bày và checklist. evidence/*.json, load-overview.png: dữ liệu/biểu đồ đo. Mã nguồn, cấu hình deploy và tests: tái lập stack PostgreSQL local, campus HTTPS, worker, backup và load profile độc lập. scripts/build-submission-report.py tái tạo PDF/biểu đồ từ evidence đã lưu.

Nguồn đối chiếu: docs/OPERATIONS.md; deploy/LOCAL_POSTGRES.md; competition/README.md; docs/NOP_DONG_THOI_VNU.md; evidence/load-run.json và source trên nhánh integrate-safe-enhancements. Repository: https://github.com/nguyenlean17/oan-tu-ti-ai-arena

Tài liệu cũ về SQLite/Vercel là profile riêng. Không thay thế hướng dẫn PostgreSQL local và không xem các giả định thiết kế là kết quả đã đo. Tên miền thực, timer Linux, hạ tầng backup độc lập và chính sách danh tính do trường cấu hình trước khi vận hành chính thức.